

ABSTRACT AND VIRTUALIZATION

What is Virtualization?

Virtualization is a technique used to use computing resources efficiently.

- Instead of directly using physical resources (like a real server, hard disk, or network cable), virtualization creates a virtual (logical) version of these resources.
- A physical resource is the actual hardware (real server, real storage).
- A virtual resource is a software-based version of that hardware.

Virtualization works by:

- Giving a logical name to a physical resource
- Connecting to the physical resource only when needed

This means the system does not permanently tie one task to one hardware device. It uses resources only when required, saving cost and improving efficiency.

1. Dynamic Mapping

Dynamic mapping means that:

- Resources are assigned dynamically, not fixed
- The system changes resource allocation based on demand

Example:

If more users join a system: Virtualization can quickly allocate more servers. If users leave: Extra servers can be released. So, the system always adjusts itself according to current needs.

2. Efficient Management

Efficient management means:

- Changes in resource allocation happen very fast
- There is no long downtime
- Operations run smoothly

Because everything is virtual:

- Servers can be added or removed in seconds
- Storage can be expanded easily
- Network settings can be changed quickly

This makes cloud systems reliable and responsive.

Examples in Real Life

1. Netflix Streaming

When millions of people watch Netflix:

- Netflix uses virtual servers
- If one server becomes overloaded: Another virtual server is instantly added
- The load (traffic) is shared between servers
- Users never notice this change because it happens automatically.

2. Google Drive

When you upload files to Google Drive:

- Your data is not stored in one physical place
- It is stored across multiple virtual storage systems
- The exact physical location is hidden from you

This helps in: Better performance, Data safety, Easy access from anywhere.

How Virtualization Works

Virtualization works by mapping virtual resources to physical resources.

- A virtual machine (VM) is mapped to real hardware
- The mapping is flexible, not fixed
- The system can change mappings quickly when conditions change

Example:

If one physical server is busy: The VM can be moved to another server. This helps in load handling and fault tolerance.

Types of Virtualization

1. Access Virtualization

Access virtualization allows users to: Access services from any device, Access from any location.

Example:

Logging into Gmail from: Phone, Laptop, Tablet. The service looks the same everywhere.

2. Application Virtualization

Application virtualization allows: Multiple users to use the same application, Each user gets a separate instance.

Example:

Online gaming servers. Each player runs the same game, but their sessions are isolated. This avoids conflicts and improves performance.

3. CPU Virtualization

CPU virtualization divides: One physical computer into multiple virtual machines. Each virtual machine: Handles specific tasks, Gets CPU resources based on demand.

Example:

A single server hosts multiple websites. Each website gets CPU power as needed.

4. Storage Virtualization

Storage virtualization: Combines multiple storage devices into one virtual storage pool. Data is often replicated (copied) for safety.

Example:

Google Drive copies your data to multiple locations. Even if one storage device fails, your data is safe.

Mobility Patterns in Virtualization

Mobility patterns describe how resources are moved or transformed.

- **P2V (Physical to Virtual):** A physical server is converted into a virtual machine. Used when moving old servers to cloud environments.
- **V2V (Virtual to Virtual):** One virtual machine is moved to another virtual environment. Used for load balancing or hardware upgrades.
- **V2P (Virtual to Physical):** A virtual machine is converted back into a physical system. Used when high performance is needed on dedicated hardware.
- **P2P (Physical to Physical):** Data is transferred between two physical systems. Traditional data migration method.
- **D2C (Datacenter to Cloud):** Resources are moved from an on-premise datacenter to the cloud. Helps reduce infrastructure cost.

- **C2C (Cloud to Cloud):** Data is moved from one cloud provider to another. Example: AWS to Google Cloud.
- **C2D (Cloud to Datacenter):** Cloud data is moved back to a physical datacenter. Used for compliance or cost control.
- **D2D (Datacenter to Datacenter):** Resources are moved between two physical datacenters. Used for backup or disaster recovery.

Key Characteristics Enabled by Virtualization (According to Gartner)

1. **Service-Based:** Users access services via interfaces. They don't care about backend details.
Example: Streaming music on Spotify. You don't know where the servers are.
2. **Scalable and Elastic:** Resources can increase or decrease automatically.
Example: During Black Friday sales: Servers scale up to handle traffic. After sale: Servers scale down.
3. **Shared Services:** Resources are shared among multiple users or applications. Improves efficiency.
Example: Multiple apps using the same server infrastructure.
4. **Metered Usage:** You pay only for what you use.
Example: AWS charges based on: Storage used, Computing power consumed.
5. **Internet Delivery:** Services are delivered over the internet. Uses standard protocols like HTTP, HTTPS. Users only need internet access.

LOAD BALANCING AND VIRTUALIZATION

What is Load Balancing?

Load balancing ensures: Requests are evenly distributed across servers. This helps to: Prevent overload, Improve speed, Increase reliability.

How Load Balancing Works

- A load balancer receives incoming requests
- It decides which server should handle the request
- Decision is based on predefined rules

Load Balancing Rules

- **Round Robin:** Requests are sent one by one. Servers are used in a circular order.
- **Fastest Response Time:** Request is sent to the server that responds fastest.
- **Least Connections:** Request goes to the server handling the fewest active users.

Network Resources That Can Be Load Balanced

- **1. Network Interfaces:** Traffic for services like DNS, FTP, HTTP is distributed
- **2. Connections:** Intelligent switches route connections to free servers
- **3. Processing:** Computational work is shared among servers
- **4. Storage:** Data is spread across multiple storage devices
- **5. Applications:** User requests are routed to different application instances

Advanced Load Balancing

Advanced load balancers act as workload managers. They make smarter decisions by: Checking server health (polling), Activating standby servers when needed, Using advanced features like: Traffic compression, Authentication, Content filtering.

Application Delivery Controller (ADC)

An ADC combines: Load balancing, Application optimization.

What ADC Does: Routes traffic based on application logic. Handles: Security, Caching, Compression. Ensures high availability.

Benefits of ADC:

- **Fault Tolerance:** If one server fails, another takes over
- **Efficiency:** Traffic is prioritized for better performance
- **Reliability:** Data is stored redundantly. Prevents data loss.

Real-Life Example: YouTube

YouTube uses ADCs to: Cache videos for faster loading, Distribute user requests efficiently, Avoid buffering and delays.

Google's Cloud and Load Balancing

Google's Cloud Infrastructure: Uses multi-layer load balancing. Handles billions of searches daily. Routes queries to the nearest datacenter. Datacenters contain clusters of servers. Each cluster performs tasks like: Caching, Indexing, Web hosting.

How Google Manages Load Balancing

- **Step 1:** User request goes to the nearest datacenter using DNS-based load balancing
- **Step 2 (Inside Datacenter):** First load balancer assigns request to a cluster. Second load balancer distributes work inside the cluster.
- **Step 3:** Servers store and process data in memory. This ensures very fast responses.

Benefits of Virtualization and Load Balancing

- **1. Fault Tolerance:** System continues working even if one server fails
- **2. Redundancy:** Data is copied to multiple locations. Prevents data loss.
- **3. Performance Optimization:** Resources are used efficiently at all times
- **4. Scalability:** System grows or shrinks based on user demand

UNDERSTANDING HYPERVISORS

Hypervisors and virtualization technology allow us to create virtual systems from physical systems. In simple words, they let one physical computer behave like many computers at the same time.

What Are Virtual Machines (VMs)?

A Virtual Machine (VM) is like a computer inside another computer. It behaves exactly like a real physical computer. It has its own: Operating system, Applications, Memory, CPU share. But it exists only as software, not as physical hardware.

From the point of view of:

- Users → it looks like a normal computer
- Applications → it behaves like a real machine

In reality, it is running on top of physical hardware using virtualization.

Real-life example:

Your laptop is one physical machine. Inside it, you run: One VM with Windows, Another VM with Linux. Even though you have one laptop, it feels like you have two computers.

How Virtual Machines Work

1. Resource Allocation

A physical computer has resources like: CPU, RAM, Storage. Virtualization cuts (slices) these resources and distributes them to VMs. Each VM gets: Its own memory, A share of CPU, Virtual disk, Virtual devices (keyboard, network card, etc.).

Example:

Physical server has: 16 GB RAM, 8 CPU cores. You can create: VM1 → 4 GB RAM, 2 CPUs; VM2 → 8 GB RAM, 4 CPUs; VM3 → 4 GB RAM, 2 CPUs.

2. Sandboxing

Sandboxing means isolation. Each VM is: Isolated from the physical system, Isolated from other VMs. If something goes wrong in one VM: Other VMs are not affected.

This is very useful for: Software testing, Running old operating systems, Cloud workload separation.

Real-life example:

You install a virus-infected app inside a VM. Only that VM is damaged. Your main computer remains safe.

3. Types of Virtual Machines

a) System Virtual Machines: Simulate a complete physical computer. Can run a full operating system. Example: Running Ubuntu Linux inside a VM on a Windows PC.

b) Process Virtual Machines: Designed to run one application or process. Do not simulate full hardware. Example: Java Virtual Machine (JVM) running Java programs.

Key Features of Virtual Machines

- **Independent Operation:** Each VM has its own OS and applications. VMs work independently like separate computers.
- **Cloning and Replication:** VMs can be Copied, Duplicated, Backed up easily. Example: Copy a VM and use it as a test system. No need to reinstall OS or software.
- **Failover Support:** If one VM fails: Another VM can take over automatically. Ensures service continuity. Example: A website VM crashes. Backup VM starts serving users immediately.

Downside of Virtual Machines

Virtualization adds performance overhead. Resources are accessed indirectly through software. Slightly slower than direct hardware access.

What Is a Hypervisor?

A Hypervisor (also called Virtual Machine Monitor – VMM) is a program that: Controls how VMs use physical hardware, Allocates CPU, memory, storage, and network, Ensures isolation between VMs. Think of it as: A manager that controls multiple virtual computers running on one physical system.

Types of Hypervisors

1. Type 1 Hypervisors (Bare-Metal Hypervisors)

- **What they are:** Installed directly on physical hardware. No host operating system required.
- **Characteristics:** Full virtualization, High performance, Low overhead.
- **Examples:** VMware ESXi, Oracle VM, Microsoft Hyper-V, Xen.
- **Key Points:** No host OS exists. OS inside VM is called guest OS. Used in enterprises and data centers.

Real-life example:

A cloud provider runs hundreds of virtual servers on physical machines using VMware ESXi.

2. Type 2 Hypervisors (Hosted Hypervisors)

- **What they are:** Installed on top of an existing operating system. Depend on the host OS for hardware access.
- **Characteristics:** Easier to install, Slightly slower, Uses paravirtualization.
- **Examples:** VMware Workstation, VirtualBox, Microsoft Virtual PC, Parallels Desktop.

- **Key Points:** Relies on host OS. Mostly used for: Learning, Development, Personal use.

Real-life example:

A student runs Linux on Windows using VirtualBox.

Paravirtualization

In paravirtualization: Guest OS communicates with host OS using para-API. Many I/O tasks are handled outside the VM. Improves performance. Hypervisors using paravirtualization: Microsoft Hyper-V, Xen (used by Amazon Web Services).

Types of Virtualization Techniques

- **1. Emulation:** VM fully simulates hardware. Guest OS does not need changes. Works across different architectures. Example: Running a PlayStation game on a PC emulator.
- **2. Paravirtualization:** Guest OS is modified. Communicates via host APIs. Faster than emulation. Used in: Microsoft Hyper-V, Xen.
- **3. Full Virtualization:** Complete hardware simulation. Guest OS does not need modification. High performance. Example: VMware ESXi in enterprises.

Application Virtual Machines

Features: Run individual applications, Slower, Highly portable, Platform independent. Cloud Usage: Not ideal for high-performance networking, Useful in parallel computing. Examples: Parallel Virtual Machine (PVM), Message Passing Interface (MPI).

Operating System Virtualization

Some OSs support OS-level virtualization. Characteristics: Creates Virtual Private Servers (VPS), All VPSs use same OS kernel, Very low overhead. Limitations: Must use same OS version. Examples: Sun Solaris Zones, IBM AIX WPARs.

VMware vSphere

VMware vSphere is a framework for managing: Servers, Storage, Networks. It pools resources to build cloud platforms.

Key Features of VMware vSphere:

- VMware vCompute: Combines servers into a resource pool
- VMware vStorage: Pools and manages storage
- VMware vNetwork: Creates virtual networks
- High Availability (HA): Keeps apps running during failures
- Fault Tolerance: Backup VM takes over automatically
- vCenter Server: Central management console

VMware Products and Tools

ESXi: Type 1 hypervisor, Installed on bare metal, Uses Linux kernel + vmkernel.

Tools:

- VMFS → Cluster file system
- VMotion → Live VM migration
- Storage VMotion → Live storage migration
- DRS → Dynamic load balancing
- DVS → Network consistency during migration

Storage and Network Virtualization

Storage Virtualization: Logical addresses mapped to physical addresses. Example: SAN uses LUNs, LBAs. Storage Types: DAS (Directly attached), Shared Storage (SAN, iSCSI, NAS).

Network Virtualization: Abstracts networking into software. Creates: VNICs, VLANs. Managed by: Hypervisors, OS, External consoles.

UNDERSTANDING MACHINE IMAGING

What is Machine Imaging?

Machine Imaging is a very important concept in cloud computing. It helps in: Portability (moving systems easily), Application deployment, System recovery.

Machine imaging works by creating a complete copy of a computer system and storing it as a single file. This file can later be reused to recreate the same system anywhere.

What is a Machine Image?

A machine image (system image) is a snapshot of a computer system that includes: Operating System (OS), Installed applications, System settings and configurations. It captures the exact state of a system at a particular moment.

What you can do with a machine image:

- Clone or copy a system: Create multiple identical systems easily
- Restore a system: Bring the system back to a previous working state
- Take snapshots: Useful for backup and troubleshooting

Simple real-life analogy:

Like taking a full backup of your phone. If something goes wrong, you restore everything exactly as it was.

Example: Amazon Machine Images (AMIs)

Amazon Web Services uses Amazon Machine Images (AMIs) to create and launch virtual machines. AMIs are pre-built system images used to launch virtual servers on AWS.

Features of AMIs:

- Pre-configured AMIs: Available with Linux, Windows. Ready to use immediately.
- Custom AMIs: Users can create their own AMIs. Include: Custom software, Custom settings, Project-specific configurations.

Types of AMIs: Free (Open-source OS images), Pay-per-use (Licensed software like Windows), Private sharing (Shared with selected users only).

Real-life example:

A startup creates one AMI with: Web server, App code. Launches 100 identical servers using the same AMI.

Porting Applications

Porting applications means: Moving software from one cloud provider or environment to another platform. Example: Moving an app from AWS to Azure.

Challenges in Porting Applications

- **Proprietary Technologies:** Cloud providers like AWS, Azure, Google Cloud use their own tools and services. This makes interoperability difficult.
- **Tight Coupling:** Applications depend on specific hardware, OS, Frameworks. Makes migration complex.

The Simple Cloud API

To solve portability issues, Zend Technologies introduced the Simple Cloud API. Goal: Provide common interfaces, Make application movement between clouds easier.

Supported Services:

- File Storage Services: Works with Amazon S3, Azure Blob Storage, Local storage.
- Document Storage Services: Supports Amazon SimpleDB, Azure Table Storage.
- Simple Queue Services: Works with Amazon SQS, Azure Queue Storage.

AppZero Virtual Application Appliance (VAA)

AppZero provides a way to move applications easily between platforms by using virtual containers.

How AppZero VAA Works:

1. **Encapsulation:** Application is packaged along with DLL files, Registry entries, Configurations. Packed into a container. Acts as an Application Image.
2. **Advantages:** Application runs in isolation. Does not modify OS files or System registry. Safer and cleaner deployment.
3. **Stateless Cloud Concept:** Application state is stored on a network share. Application can run on any cloud, any system. No dependency on local machine state.

Key Tools in AppZero: AppZero Creator Wizard (Creates the VAA), AppZero Admin Tool (Manages applications), AppZero Director (Installs and runs applications), AppZero Dissolve (Removes VAA layer, Installs app directly on OS if required).

CAPACITY PLANNING IN CLOUD COMPUTING

Why Capacity Planning?

At first, cloud seems: Unlimited, Always available. But in reality: Resources are limited, Costs increase if unmanaged.

Capacity planning ensures: Enough resources for demand, Cost-effective usage.

Key Concepts of Capacity Planning

1. Capacity Planning vs System Optimization

- **System Optimization:** Uses existing resources better, Improves efficiency.
- **Capacity Planning:** Finds system limits, Adds resources to meet future demand. Even if inefficiencies exist: Capacity planning ensures demand is met.

2. Iterative Process of Capacity Planning

Capacity planning is continuous. Step-by-step process: Assess current system, Measure workloads (CPU, RAM, Disk, Network), Test overload limits, Predict future demand using past data, Adjust resources (add/remove), Repeat regularly.

Defining Baselines and Metrics

Baseline = current system performance. Used as reference for future planning. Metrics help measure: Performance, Bottlenecks, Growth needs.

LAMP Stack Example

What is LAMP?

LAMP stands for:
Linux → Operating system
Apache → Web server
MySQL → Database
PHP → Scripting language

LAMP on AWS Example: A LAMP-based system may include: Apache web server (Handles page views/hits per sec), MySQL database (Handles queries/sec).

Metrics to Monitor: Page Views (Hits per second), Database Transactions (Queries per second).

Traffic Analysis

Traffic is analyzed using: Daily logs, Weekly logs, Yearly logs. Example: Traffic spikes at 10 AM EST and 1 PM EST. Capacity planners: Match spikes to events, Predict future growth (e.g., traffic doubling every year).

Baseline Metrics for Capacity Planning

- **WT (Total Workload):** Total amount of work in a given time
- **WAVG (Average Workload):** Average workload over multiple time periods

These metrics help decide: When to scale up, When to scale down.

System Metrics for Machine Instances

In cloud computing, machine instances (virtual servers) are measured using system metrics. These metrics help us understand how well a server is performing and where problems may occur.

1. **CPU (Processor Utilization):** Measures how much of the processor is being used. High CPU usage means system is doing heavy computation or is overloaded. Example: CPU at 90% → server is under heavy load.
2. **Memory (RAM):** Measures how much RAM is being used. If memory usage is high: System may slow down, Swapping to disk may occur.
3. **Disk (I/O Operations):** Measures how fast data is Read from/Written to disk. Disk I/O bottlenecks slow down databases and file systems.
4. **Network:** Measures Bandwidth usage, Packet transmission. Important for web servers and APIs.

Tools for Monitoring Metrics

Monitoring tools help collect, visualize, and analyze system metrics.

- **Linux Monitoring Tools:** sar (sysstat package) for historical analysis. RRDTool for time-based performance data graphs.
- **Windows Monitoring Tools:** Task Manager (Real-time usage), Performance Monitor (Advanced logging and alerts).
- **Cloud Monitoring Tools:** Amazon CloudWatch (Monitors AWS resources, Supports alarms and automated actions).

Load Testing

Load testing checks how a system behaves under increasing load. Simulates: Many users, Heavy traffic. Helps find breaking points.

Questions Answered by Load Testing: Maximum Load, Bottleneck Identification, Configuration Improvement, Performance Comparison.

Common Load Testing Tools: Basic Tools (HTTPPerf, Siege, Autobench), Advanced Load Testing Tools (JMeter, LoadRunner, OpenSTA).

Resource Ceilings and Bottlenecks

A resource ceiling is the maximum limit of a resource before performance degrades.

Example: Web Server Bottleneck

Network I/O reaches 100% utilization. Happens at only 50% of expected load. Network becomes the bottleneck. Solutions: Scale out, Add more servers, Upgrade network, Faster links.

Database Resource Ceilings

- 1. Disk I/O:** Database speed improves with faster disks, more disk spindles, SSDs.
- 2. Master / Slave Database Architecture:** Master Server handles WRITE operations. Slave Servers handle READ operations. **Replication Bottleneck:** Writes overwhelm replication. Slaves cannot keep up. Causes lag and inconsistency.

Advanced Capacity Planning

- 1. Performance Consoles:** Tools like Microsoft Management Console (MMC), Amazon CloudWatch. They display performance graphically, Track KPIs, Help in decision-making.
- 2. Simulations vs Real-World Data:** Simulations are useful, But real-world data is more accurate. Actual user behavior reveals true bottlenecks.

Overall System Capacity (WT)

WT = Total workload across all servers.

Formula: $WT = \sum (WSnP + WSnV)$

Where:

WSnP → Physical server workload

WSnV → Virtual server workload

Overhead Considerations: Extra capacity kept for traffic spikes and unexpected demand. Depends on risk tolerance and response time for scaling.

Scaling in Capacity Planning

When resources hit ceilings, scaling is required. Two approaches:

1. Vertical Scaling (Scale-Up)

- **Definition:** Add more power to the same machine. Examples: Upgrade CPU, Add more RAM.
- **Best Used For:** CPU-intensive tasks, Memory-heavy applications, Rendering, In-memory databases.
- **Limitations:** Expensive, Creates single powerful machines, Harder to manage at scale.

2. Horizontal Scaling (Scale-Out)

- **Definition:** Add more servers to share load. Examples: Add servers to a web cluster.
- **Best Used For:** I/O-heavy applications, Web servers, Distributed systems.
- **Advantages:** Better resource pooling, High availability, Fault tolerance.

MySQL Database Resource Ceilings

Database systems also face resource limits (ceilings). When these limits are reached, performance degrades.

1. Master–Slave Setup

Master database handles all WRITE operations (INSERT, UPDATE, DELETE). Slave databases handle READ operations (SELECT queries). This setup improves performance by separating read and write workloads.

Problem: Replication Bottleneck. High WRITE traffic on the master overloads the master server. Slaves cannot replicate data fast enough. Result: Replication lag. Slaves show outdated data.

Solution:

Use more powerful disk arrays. Use faster interconnects between master and slaves.

2. Scaling Database Systems

- **For Smaller Applications:** Use master–slave architecture. Simple and cost-effective. Works well for Blogs, Small e-commerce sites.
- **For Larger Applications:** Use federated databases. Workload is split across multiple databases. Handles High transaction volumes, Large datasets.

Server and Instance Types in Cloud Computing

1. Standardization

Using identical hardware and software configurations. Benefits: Easier capacity planning, Predictable performance, Faster troubleshooting, Simpler scaling.

2. Amazon Machine Instances (AMIs / Instance Types)

Cloud providers offer different instance types for different workloads.

- **Micro Instance:** Very small memory and CPU. Supports short CPU bursts. Used for Testing.
- **Small Instance (m1.small):** Moderate I/O performance. Suitable for Web applications.
- **High-Memory Quadruple Extra Large (m2.4xlarge):** Very large memory. Suitable for In-memory databases.
- **High-CPU Extra Large (c1.xlarge):** Optimized for CPU. Used for Scientific computing, Rendering.

3. Considerations for Cloud Instances

Variability: Cloud performance can change. Reason: Underlying infrastructure, Multi-tenant environment.

Scaling Options: Use Amazon Auto Scaling. Automatically Add/Remove instances based on load.

Network Capacity in Cloud Systems

Network capacity is a critical factor in cloud performance.

Key Factors Affecting Network Capacity: Server Interface Traffic, WAN Traffic Inside the Cloud, Cloud ↔ Local ISP Traffic.

Network Challenges: Measuring WAN Traffic is hard. Last-Mile Bottleneck (between ISP and user) is often the biggest issue.

Network Solutions: Use Monitoring Tools like PathViewCloud. Advocate better broadband infrastructure.

Final Summary (Conceptual)

- Databases hit ceilings due to disk, network, or replication limits
- Monitoring tools identify bottlenecks
- Standardized servers simplify scaling
- Network capacity often limits cloud performance

- Smart scaling and monitoring ensure reliability